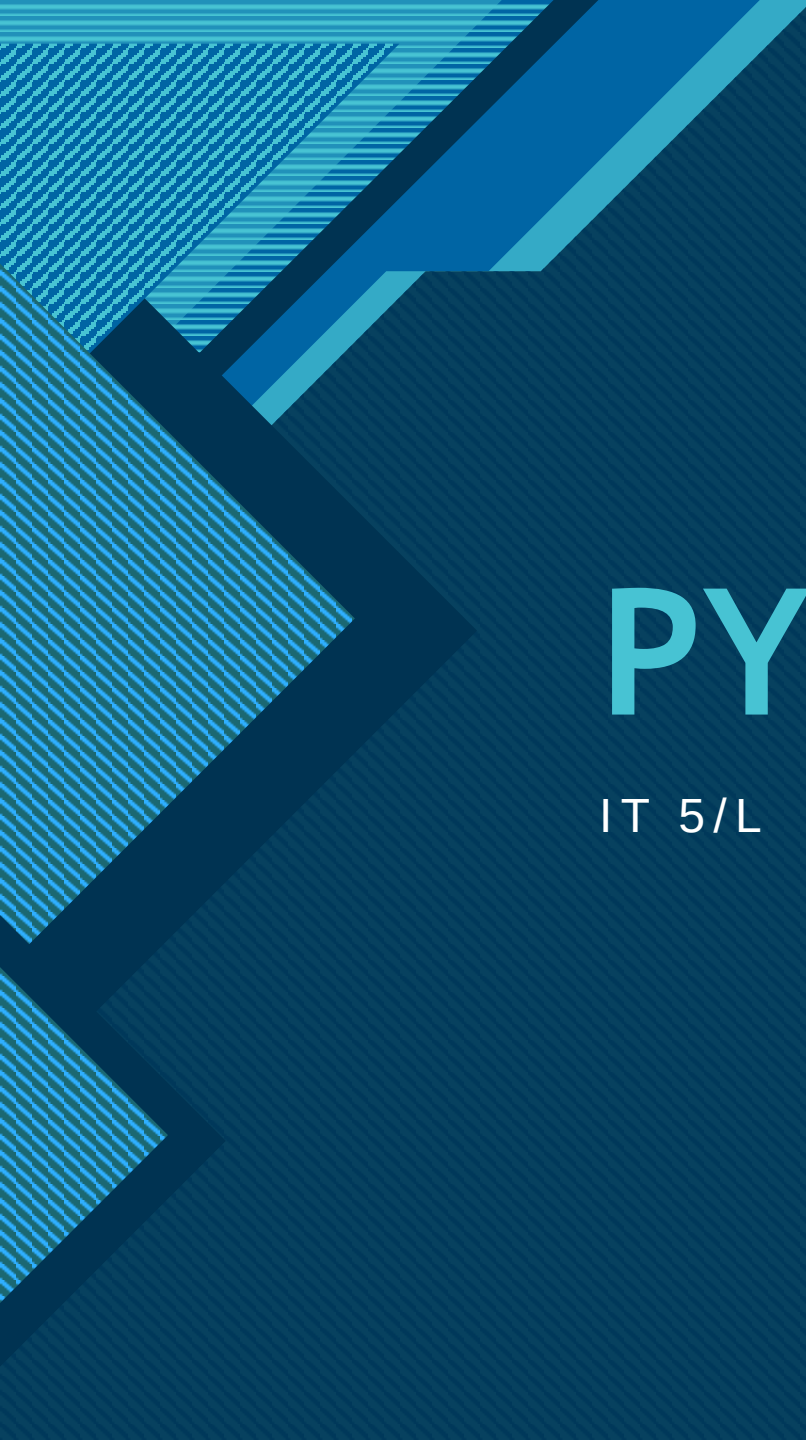




PYTHON OPERATORS

IT 5/L

Python Operators

In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for the purpose of logical and arithmetic operations.

Types of Operators in Python

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operator
5. Bitwise Operator
6. Identity Operator
7. Membership Operators

Python Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations.

Operator	Name	Example (x=5, y=2)	Output
+	Addition	$x + y$	7
-	Subtraction	$x - y$	3
*	Multiplication	$x * y$	10
/	Division	x / y	2.5
%	Modulus	$x \% y$	1
**	Exponentiation	$x ** y$	25
//	Floor Division	$x // y$	2

EXAMPLE USING ARITHMETIC OPERATORS

- In the example below, we use the + operator to add together two values:

```
x = 100 + 50;
```

- Although the + operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable:

```
sum1 = 100 + 50 // 150 (100 + 50)
```

```
sum2 = sum1 + 250; // 400 (150 + 250)
```

```
sum3 = sum2 + sum2; // 800 (400 + 400)
```

Python Assignment Operators

Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **=	x = x ** 3

Python Comparison Operators

Comparison operators are used to compare two values:

Operator	Name	Example
<code>==</code>	Equal	<code>x == y</code>
<code>!=</code>	Not equal	<code>x != y</code>
<code>></code>	Greater than	<code>x > y</code>
<code><</code>	Less than	<code>x < y</code>
<code>>=</code>	Greater than or equal to	<code>x >= y</code>
<code><=</code>	Less than or equal to	<code>x <= y</code>

Python Logical Operators

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverse the result, returns False if the result is true	<code>not(x < 5 and x < 10)</code>

Truth Table - Logical And

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Truth Table - Logical Or

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Truth Table - Logical Not

A	!A
True	False
False	True

Python Bitwise Operators

Bitwise operators are used to perform binary logic with the bits of an integer or long integer. It is used to perform manipulation of individual bits of a number.

Operator	Name	Description	Example
&	AND	Sets each bit to 1 if both bits are 1	$x \& y$
	OR	Sets each bit to 1 if one of two bits is 1	$x y$
~	NOT	Inverts all the bits	$\sim x$
^	XOR	Sets each bit to 1 if only one of two bits is 1	$x \wedge y$
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	$x \ll 2$
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	$x \gg 2$

Python Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

is Operator

```
x = ["apple", "banana"]
```

```
y = ["apple", "banana"]
```

```
z = x
```

```
print(x is z)
```

```
# returns True because z is the same object as x
```

```
print(x is y)
```

```
# returns False because x is not the same object as y, even if they have  
the same content
```

```
print(x == y)
```

```
# to demonstrate the difference between "is" and "==" : this comparison  
returns True because x is equal to y
```

Output

True
False
True

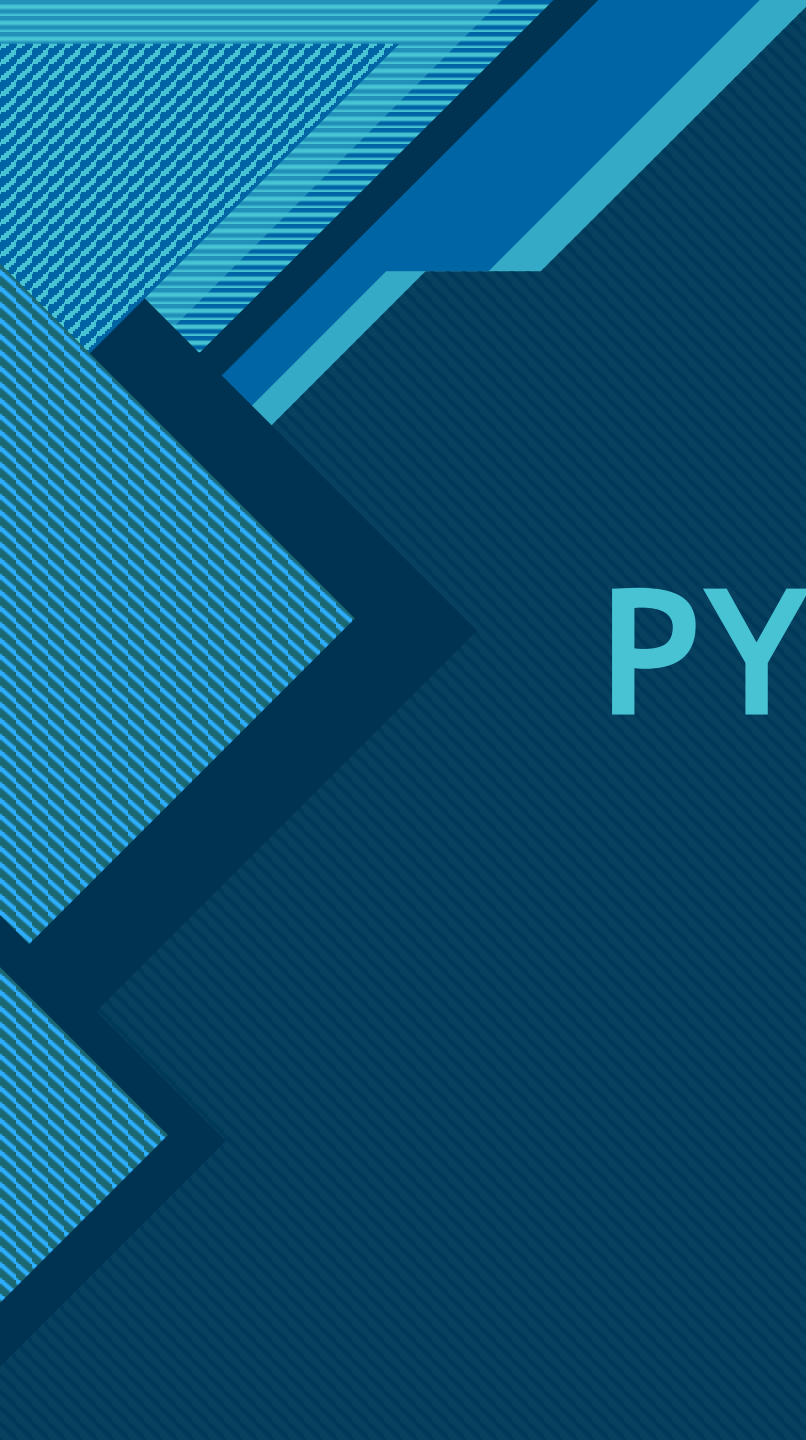
Python Membership Operators

Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example	
in	Returns True if a sequence with the specified value is present in the object	x in y	<pre>x = ["apple", "banana"] print("banana" in x)</pre>
not in	Returns True if a sequence with the specified value is not present in the object	x not in y	<pre>x = ["apple", "banana"] print("pineapple" not in x)</pre>

Operator Precedence

- All operators inside the parenthesis are evaluated first.
- $*$, $/$ and $\%$ are at the same level of precedence and are evaluated next.
- $+$ and $-$ have the same level and are evaluated last.
- When operators are at the same level, we use left to right associativity.
- $3*7-6+2/4+6$ means $((3*7)-6) + ((2*5)/4) + 6$.



PYTHON STRINGS

Python Strings

Strings represent text. In python, Strings are surrounded by either single quotation marks, or double quotation marks.

For example:

'hello' is the same as "hello".

You can display a string literal with the print() function:

```
a = "Hello"  
print(a)
```

Output:



Python Strings

Strings are Arrays

- Like many other popular programming languages, strings in Python are arrays of bytes representing unicode characters.
- However, Python does not have a character data type, a single character is simply a string with a length of 1.
- Square brackets can be used to access elements of the string.

```
a = "Hello, World!"  
print(a[1])
```

Python Strings

String length

To get the length of a string, use the **len()** function.

```
a = "Hello, World!"  
print(len(a))
```

Output:

13

Python Strings

Check String

To check if a certain phrase or character is present in a string, we can use the keyword **in**.

```
txt = "The best things in life are free!"  
print("free" in txt)
```

Output:

```
true
```

Python Strings

Escape Characters

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash \ followed by the character you want to insert. An example of an illegal character is a double quote inside a string that is surrounded by double quotes:

```
#You will get an error if you use double quotes inside a string that are  
surrounded by double quotes:  
txt = "We are the so-called "Vikings" from the north."
```

Output:

```
File "demo_string_escape_error.py", line 1  
    txt = "We are the so-called "Vikings" from the north."  
                                   ^  
SyntaxError: invalid syntax
```

Escape Characters

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value

ACTIVITY

Python offers a set of built-in methods for manipulating strings, known as **String Methods**. In this activity, give at least 20 examples of String Methods. For each method, include its definition and demonstrate how it is used. Please follow the format below:

1. `capitalize()` – The `capitalized()` method returns a string where the first character is upper case, and the rest is lower case.

Example:

```
txt = "python is FUN!"  
  
x = txt.capitalize()  
  
print (x)
```

Output:

```
Python is fun!
```